Analyse des avis clients en assurance

Groupe : Data&IA3 A5

Nom : GUMIENIAK Lukasz

Sommaire

1 | Présentation générale du projet et des données

1.1 | Présentation de la problématique

1.2 | Présentation des données

1.3 | Exploration et nettoyage des données

2 | Embedding et modèle

2.1 | Embedding pour identifier des mots similaires

2.2 | Modèle de prédiction de la note

3 | Streamlit Application

3.1 | Mise en place de l'application

3.2 | Fonctionnement de l'application

Objectif et contexte

- **Contexte :**
  - Des enquêtes de satisfaction ont été réalisées auprès des clients, une analyse des avis libres sur différents assureurs et produits.

- **Objectifs :**
  - Prédire la note d'un avis
  - Identifier la thématique d'un avis

- **Techniques utilisées :**
  - Apprentissage supervisé
  - Techniques de Traitement automatique du langage naturel

1 | Présentation de la problématique et des données du projet

1.1 Présentation de la problématique

- Comprendre les produits et les assureurs ainsi que leurs notes moyennes
- Savoir prédire la note d'un avis, qu'il soit en français ou en anglais

- Problématique : Comment prédire la note d'un avis, en fonction de l'assureur, du produit ainsi que de la langue (français ou anglais) et en faire une application ?

- Problème de régression ⇒ prédiction d'une note

1.2 Présentation des données

- **35 fichiers d'avis différents en format Excel qu'il fallait regrouper en un seul Dataframe pour l'étude**

Import des données

Entrée [2]:

```
# Création d'une liste vide pour stocker les dataframes
dfs = []

# Parcourir tous les fichiers xlsx dans le répertoire courant
for file in os.listdir():
    if file.endswith('.xlsx'):
        # Lire chaque fichier xlsx et l'ajouter à la liste
        df_temp = pd.read_excel(file)
        dfs.append(df_temp)

# Concaténer tous les dataframes en un seul
df_final = pd.concat(dfs, ignore_index=True)
```

Entrée [3]:
```
df_final.head(10)
```

Out[3]:

	note	auteur	avis	assureur	produit	type	date_publication	date_exp	avis_en	avis_cor	avis_cor_en
0	4.0	audurier-c-136272	La personne au téléphone était Clair et sympat...	L'olivier Assurance	auto	train	06/10/2021	01/10/2021	The person on the phone was clear and friendly...	NaN	NaN
1	4.0	paul-a-122970	Satisfait.\n\nRéactivité, simplicité. Prix att...	APRIL Moto	moto	train	09/07/2021	01/07/2021	Satisfied.\n\nReactivity, simplicity. Attracti...	NaN	NaN
2	1.0	kitty-38517	Assureur à fuir, n assure pas ses responsabili...	SwissLife	vie	train	15/10/2020	01/10/2020	Insurer to flee, does not ensure its responsib...	NaN	NaN
3	1.0	laure97134-87907	Voilà 3 mois que la GMF me fait attendre pour ...	GMF	habitation	train	03/03/2020	01/03/2020	The GMF has been waiting for a water damage fo...	NaN	NaN
4	3.0	bourouane-l-129916	Je suis bien avec cet assurance.elle est prati...	L'olivier Assurance	auto	train	28/08/2021	01/08/2021	I am good with this insurance. She is practica...	NaN	NaN
5	5.0	ilyas-k-115898	Le drive box nest pas tres fiable.Ca demande b...	Direct Assurance	auto	train	03/06/2021	01/06/2021	The Drive Box is not very reliable.ca requests...	NaN	NaN
6	5.0	nadette-121889	J'ai malheureusement eu à faire appel à plusie...	Pacifica	auto	train	30/06/2021	01/06/2021	Unfortunately, I had to call on Pacifica for b...	NaN	NaN
7	4.0	mimile-81147	Suite à mon bris de glace le traitement de cel...	Pacifica	auto	train	20/11/2019	01/11/2019	Following my ice broken the treatment of it wa...	NaN	NaN
8	1.0	rahma-63689	Bonjour , Actuellement je suis chez Active Ass...	Active Assurances	auto	train	01/05/2018	01/05/2018	Hello, currently I am at Active Insurance unde...	NaN	NaN
9	1.0	bichon-70941	Ma mère est décédée depuis plusieurs mois et ...	SwissLife	vie	train	04/02/2019	01/02/2019	My mother has died for several months and sti...	NaN	NaN

34435 observations
11 variables

Note : Note de l'auteur allant de 1 à 5
Auteur : Auteur de l'avis
Avis : Avis de l'auteur (en français)
Assureur : Assurance concerné par l'avis
Produit : Type de produit assuré (assurance auto, habitation …)
Type : Savoir s'il s'agit du jeu d'entraînement ou de test
date_publication : Date de publication de l'avis
avis_en : Avis de l'auteur (traduit en anglais)

Analyse des types de variables

Entrée [5]:
```
df_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34435 entries, 0 to 34434
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   note              24104 non-null  float64
 1   auteur            34434 non-null  object
 2   avis              34435 non-null  object
 3   assureur          34435 non-null  object
 4   produit           34435 non-null  object
 5   type              34435 non-null  object
 6   date_publication  34435 non-null  object
 7   date_exp          34435 non-null  object
 8   avis_en           34433 non-null  object
 9   avis_cor          435 non-null    object
 10  avis_cor_en       431 non-null    object
dtypes: float64(1), object(10)
memory usage: 2.9+ MB
```

Les colonnes `date_publication` et `date_exp` sont actuellement au format `object`, alors qu'elles devraient être au format date. Il est donc nécessaire de convertir ces deux colonnes dans le format approprié.

1.3 Exploration et nettoyage des données

- Appliquer tous les traitements de nettoyage nécéssaires
- Explorer davantage nos données

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34435 entries, 0 to 34434
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   note              24104 non-null  float64
 1   auteur            34434 non-null  object
 2   avis              34435 non-null  object
 3   assureur          34435 non-null  object
 4   produit           34435 non-null  object
 5   type              34435 non-null  object
 6   date_publication  34435 non-null  datetime64[ns]
 7   date_exp          34435 non-null  datetime64[ns]
 8   avis_en           34433 non-null  object
 9   avis_cor          435 non-null    object
 10  avis_cor_en       431 non-null    object
dtypes: datetime64[ns](2), float64(1), object(8)
memory usage: 2.9+ MB
```

Premier traitement :
Avoir les bons formats pour chacune de nos variables (traitement appliqué sur les variables de dates afin qu'elles aient bien un format de date)

Entrée [6]:

```python
# Mettre les dates sous format date

df_final['date_publication'] = pd.to_datetime(df_final['date_publication'], format='mixed', errors='coerce')
df_final['date_exp'] = pd.to_datetime(df_final['date_exp'], format='mixed', errors='coerce')
```

Analyse des valeurs manquantes

Entrée [8]:

```
print("\nPourcentage de valeurs manquantes par colonne :")
for column in df_final.columns:
    pct_missing = (df_final[column].isnull().sum() / len(df_final)) * 100
    print(f"{column}: {pct_missing:.2f}%")
```

```
Pourcentage de valeurs manquantes par colonne :
note: 30.00%
auteur: 0.00%
avis: 0.00%
assureur: 0.00%
produit: 0.00%
type: 0.00%
date_publication: 0.00%
date_exp: 0.00%
avis_en: 0.01%
avis_cor: 98.74%
avis_cor_en: 98.75%
```

Entrée [9]:

```
print("\nRépartition des types pour les notes manquantes :\n",
      df_final[df_final['note'].isna()]  # Filtrer les lignes où note est vide
      ['type']                           # Sélectionner la colonne type
      .value_counts(normalize=True)      # Compter les occurrences en proportion
      .mul(100).round(2),               # Convertir en pourcentage
      "%")
```

```
Répartition des types pour les notes manquantes :
 type
test    100.0
Name: proportion, dtype: float64 %
```

Conclusion et traitement

La colonne `note` contient 30% de valeurs manquantes, car il s'agit de la base de test. Cela signifie que ces valeurs manquantes sont normales et qu'aucun traitement ne sera effectué sur cette colonne.

Les étapes suivantes seront suivies pour traiter les valeurs manquantes :

1. Supprimer les colonnes `avis_cor` et `avis_cor_en`, qui contiennent respectivement 98,74% et 98,75% de valeurs manquantes.
2. Supprimer les observations où la colonne `avis_en` présente des valeurs manquantes, car cette colonne ne contient que 0,01% de valeurs manquantes.
3. Séparer la base de données en ensembles d'apprentissage et de test.

Entrée [8]:

```
# 1. Supprimer les colonnes avec trop de valeurs manquantes
df_clean = df_final.drop(['avis_cor', 'avis_cor_en'], axis=1)
print("\nColonnes supprimées. Nouvelles dimensions :", df_clean.shape)

# 2. Supprimer les observations où avis contient des valeurs manquantes
df_clean = df_clean.dropna(subset=['avis_en'])
print("Observations après suppression des avis manquants :", len(df_clean))

# 3. Séparation en ensembles d'apprentissage et de test
df_test = df_clean[df_clean['note'].isna()].copy()
df_train = df_clean[~df_clean['note'].isna()].copy()
```

```
Colonnes supprimées. Nouvelles dimensions : (34435, 9)
Observations après suppression des avis manquants : 34433
```

```
Dimensions des ensembles :
Apprentissage : 24103 observations
Test : 10330 observations

Proportion apprentissage/test :
Apprentissage : 70.00%
Test : 30.00%

Distribution des types dans l'ensemble d'apprentissage :
type
train    100.0
Name: proportion, dtype: float64 %

Distribution des types dans l'ensemble de test :
type
test    100.0
Name: proportion, dtype: float64 %
```

Par la suite nous allons exporter 4 fichiers différents (et nous allons passer sur l'environnement Colab notamment pour construire et entraîner notre modèle) :

- **df_train.xlsx**, notre base d'apprentissage sur laquelle nous allons entraîner notre modèle et effectuer nos différentes visualisations
- **df_train.xlsx**, notre base de test sur laquelle nous allons tester notre modèle
- **modalites_assureurs.xlsx**, un fichier contenant toutes les modalités de la variable assureur (pour le sélecteur sur l'appli)
- **modalites_produit.xlsx**, un fichier contenant toutes les modalités de la variable produit (pour le sélecteur sur l'appli également)

```
Analyse des n-grammes fréquents...

Bigrammes français les plus fréquents :
{'! !': 9689, 'je suis': 8147, 'de la': 5171, '. je': 4959, ', je':

Bigrammes anglais les plus fréquents :
{'! !': 9386, 'i am': 9269, '. i': 8043, 'of the': 7181, ', i': 7168

Trigrammes français les plus fréquents :
{'! ! !': 5721, 'je suis satisfait': 2030, "m ' a": 1801, 'satisfait

Trigrammes anglais les plus fréquents :
{'! ! !': 5530, 'satisfied with the': 3031, 'i am satisfied': 2815,
```

Analyse des n-grammes (bigrammes et trigrammes ici) sur le dataset **df_train** (en français et anglais):

- On remarque que les résultats sont biaisés dû aux différentes ponctuations (« ! », « , » ou bien « ? ») ainsi que différents « stopwords » contraignant qui empêche de déceler les bigrammes et trigrammes essentiels

- Solution : créer deux nouvelles variables d'avis nettoyées (une pour la langue française, l'autre pour la langue anglaise), en éliminant les ponctuations, les caractères numériques ainsi que les stopwords (définis au préalable avec la bibliothèque 'nltk')

```python
# Création des listes de stopwords
stop_words_fr = set(stopwords.words('french'))
stop_words_en = set(stopwords.words('english'))

def is_french(text):
    """
    Détecte si un texte est en français en utilisant langdetect
    Args:
        text: texte à analyser
    Returns:
        bool: True si le texte est en français, False sinon
    """
    try:
        return detect(text) == 'fr'
    except:
        return False

def clean_text(text, is_french=True):
    """
    Nettoie le texte en enlevant la ponctuation, les stopwords et les caractères numériques
    Args:
        text: texte à nettoyer
        is_french: True si le texte est en français, False si en anglais
    """
```

```python
# Application du nettoyage sur les colonnes
print("\nNettoyage des avis français...")
df_train['avis_cleaned'] = df_train['avis'].apply(lambda x: clean_text(x, is_french=True))

print("Nettoyage des avis anglais...")
df_train['avis_en_cleaned'] = df_train['avis_en'].apply(lambda x: clean_text(x, is_french=False))
```

```
Bigrammes français les plus fréquents :
{'cette assurance': 1742, 'service client': 1552, 'direct assurance': 1359, 'satisfait service': 1278, 'prise charge': 1018, 'a fuir': 813,

Bigrammes anglais les plus fréquents :
{'customer service': 1680, 'satisfied service': 1477, 'direct insurance': 1255, 'recommend insurance': 490, 'insurance company': 473, 'car i

Trigrammes français les plus fréquents :
{'rapport qualité prix': 383, 'depuis plus ans': 373, 'satisfait service prix': 281, 'chez direct assurance': 237, 'tout va bien': 181, 'bon

Trigrammes anglais les plus fréquents :
{'mutual insurance company': 248, 'satisfied service prices': 232, 'good value money': 175, 'recommend direct insurance': 167, 'satisfied se
```

Grâce au nettoyage effectué notamment avec les deux nouvelles variables (« **avis_cleaned** » pour la langue française et « **avis_en_cleaned** » pour la langue anglaise), nous constatons maintenant des bigrammes et trigrammes beaucoup mieux représenter, qui nous laisse penser que notre nettoyage a été efficace.

Nous allons donc garder ces deux champs pour la suite, notamment pour entraîner notre modèle afin qu'il cible du mieux possible les mots importants.

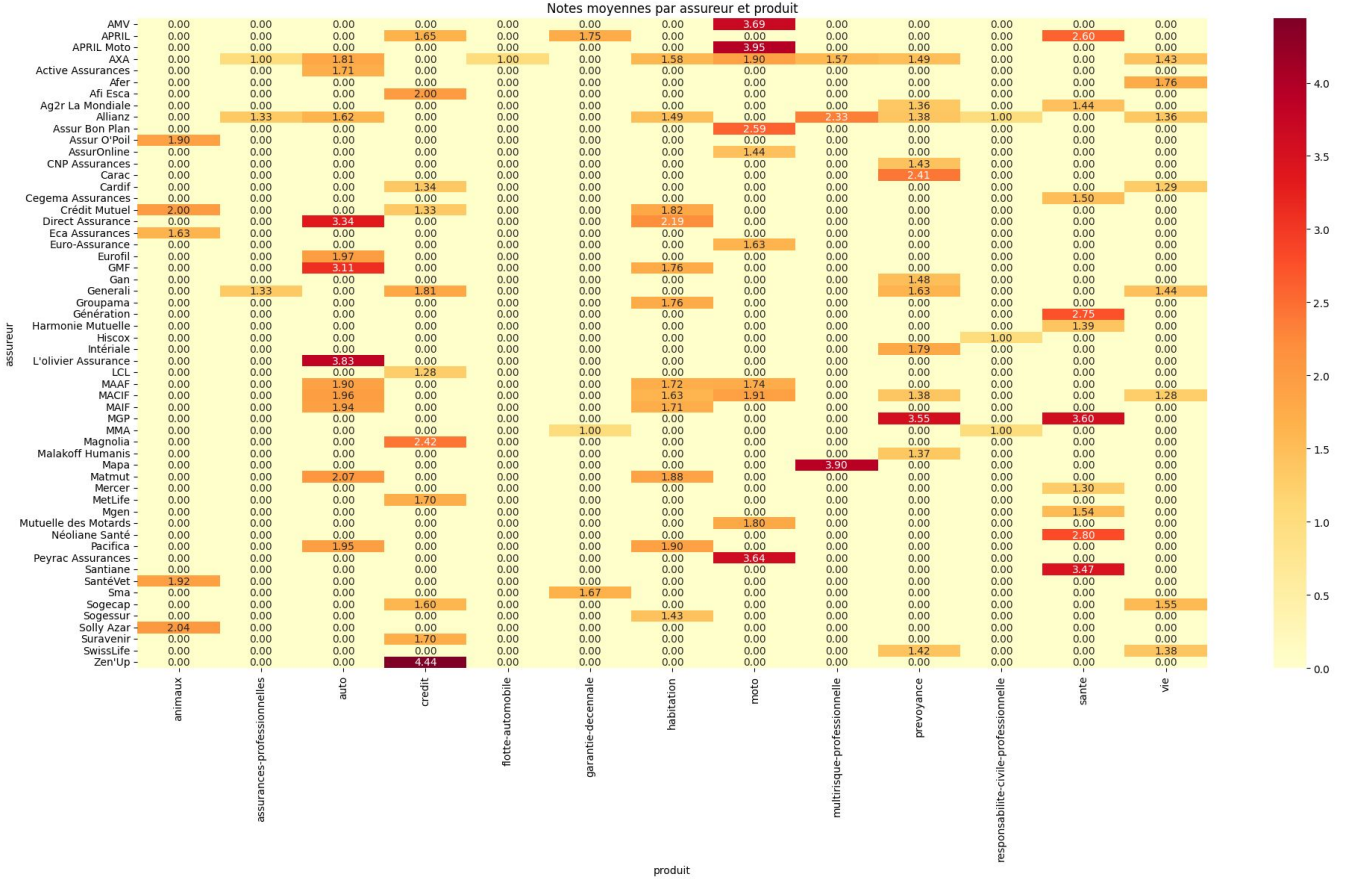
```
=== NOTE 5 ===

Mots français les plus fréquents :
'très': 3650 occurrences
'assurance': 2237 occurrences
'prix': 2149 occurrences
'a': 1873 occurrences
'service': 1677 occurrences
'satisfait': 1665 occurrences
'rapide': 1101 occurrences
'bien': 1043 occurrences
'merci': 994 occurrences
'recommande': 974 occurrences
'satisfaite': 730 occurrences
'tout': 718 occurrences
'simple': 714 occurrences
'plus': 590 occurrences
'bon': 569 occurrences
'cette': 516 occurrences
'écoute': 509 occurrences
'chez': 504 occurrences
'contrat': 485 occurrences
'client': 454 occurrences
'bonne': 454 occurrences
'efficace': 445 occurrences
'tarifs': 438 occurrences
'tres': 427 occurrences
'qualité': 423 occurrences
'site': 422 occurrences
'direct': 408 occurrences
'olivier': 399 occurrences
'services': 386 occurrences
'tarif': 378 occurrences

Mots anglais les plus fréquents :
'satisfied': 2397 occurrences
'insurance': 2200 occurrences
'service': 1677 occurrences
'price': 1496 occurrences
'recommend': 1297 occurrences
'good': 1146 occurrences
'prices': 1112 occurrences
'thank': 996 occurrences
'well': 775 occurrences
'fast': 769 occurrences
'simple': 643 occurrences
'customer': 492 occurrences
'services': 486 occurrences
'contract': 473 occurrences
'easy': 426 occurrences
'direct': 415 occurrences
'clear': 407 occurrences
'phone': 384 occurrences
'everything': 378 occurrences
'advisor': 358 occurrences
'listening': 342 occurrences
'always': 336 occurrences
'speed': 334 occurrences
'site': 322 occurrences
'time': 321 occurrences
'attractive': 315 occurrences
'years': 303 occurrences
'subscription': 298 occurrences
'quick': 295 occurrences
'see': 292 occurrences
```

On a également cherché à savoir quels sont les mots les plus fréquents par note, ce qui peut être un bon indicateur pour générer nos avis sur l'application.

Ici, nous avons pris l'exemple des mots anglais et français qui étaient le plus représentés avec une note de 5 renseigné par l'auteur. On y retrouve sans surprise des mots comme « très », « satisfait », « qualité » ou bien « recommande » et la même chose du côté anglais.



Pour finir cette partie, nous avons voulu représenter les notes moyennes par assureurs en fonction du produit. Nous voyons beaucoup de variations (ex : **LCL** plus connu du grand public avec une note moyenne pour les **crédits** de **2,42** tandis que **Zen'Up** a une moyenne de **4,44** pour le même produit)

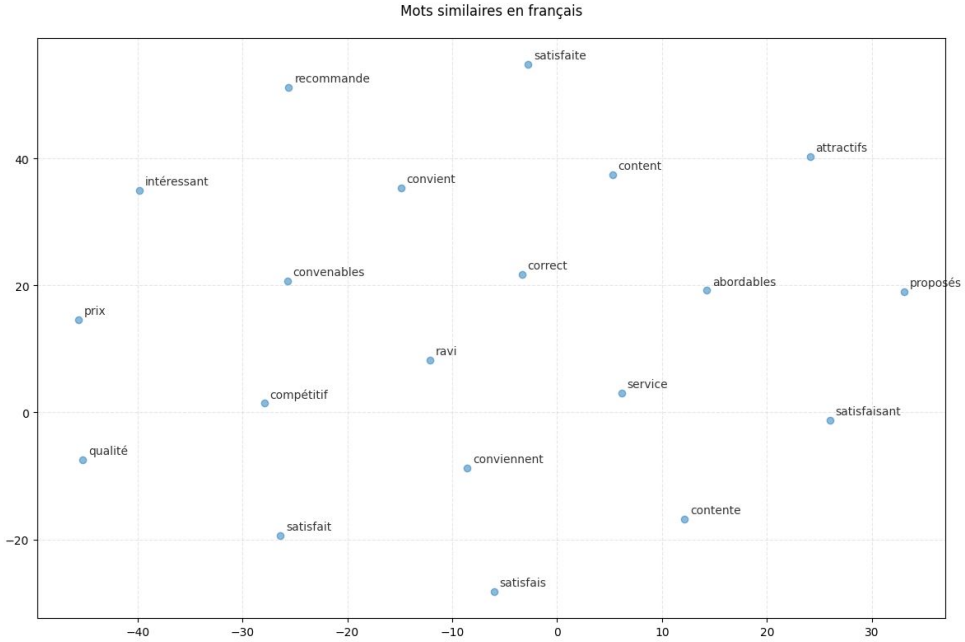
Le but de notre modèle va donc d'être de s'adapter non seulement aux mots contenus dans l'avis mais également de faire sa prédiction en fonction de l'assureur et du produit.

2 Embedding et modèle

2.1 Embedding pour identifier des mots similaires

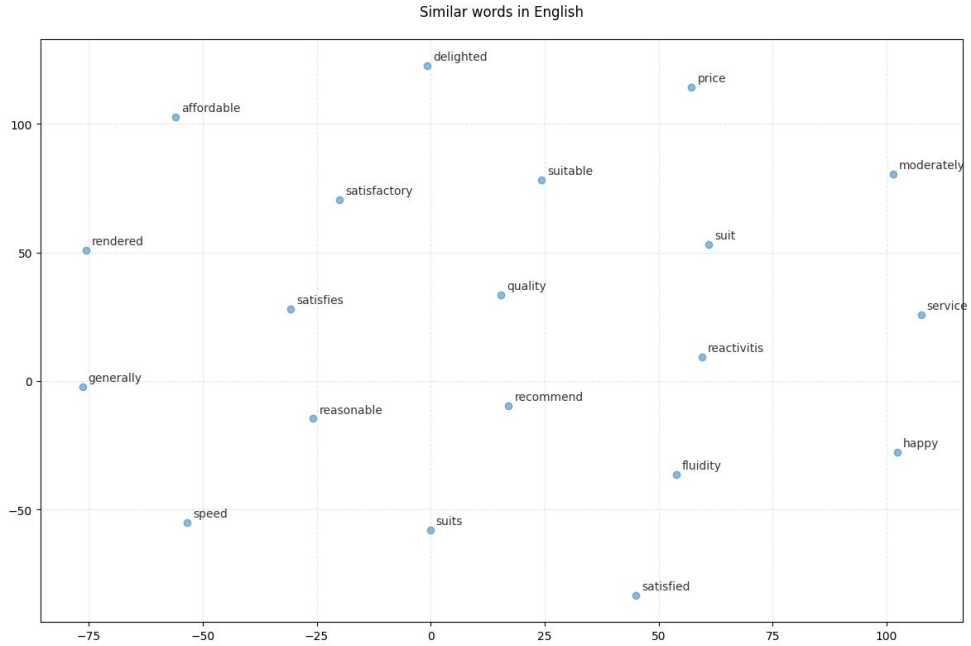
Analyse sémantique des mots :

- **Préparation et entraînement de deux modèles Word2Vec (un pour les mots français, l'autre pour les mots anglais)**
- **Recherche des mots similaires avec la méthode most_similar de Word2Vec (qui utilise implicitement la similarité cosine)**
- **Implémentation d'une recherche sémantique : à partir de plusieurs mots, identifier ceux qui ont un concept similaire**
- **Visualisation avec Matplotlib**



On remarque :

- Un groupe de mots autour de la satisfaction : "satisfait", "satisfaite", "satisfais", "satisfaisant«
- Un groupe lié aux prix : "prix", "compétitif", "abordables", "convenables"
- Un groupe sur la qualité du service : "service", "qualité", "correct", "convient"
- Les mots proches dans le graphique ont des significations similaires et des utilisations communes comme « qualité » et « prix » ou bien « attractifs » et « proposés », liés probablement par la catégorie des offres



Nous retrouvons un cas similaire pour les mots en anglais avec :

- Des groupements similaires sur la qualité de service, du prix et de la satisfaction
- Les mots traduits occupent des places relativement similaires (ex : prix/price, service/service, qualité/quality)

```
=== Recherche sémantique ===

Recherche en français:

Résultats pour 'service client':
- déplorable: 0.7918
- inexistant: 0.7762
- médiocre: 0.7416
- clientèle: 0.7391
- joignable: 0.7358

Résultats pour 'prix élevé':
- franchises: 0.8877
- marché: 0.8840
- élevées: 0.8673
- tarif: 0.8601
- légèrement: 0.8330

Résultats pour 'bonne qualité':
- excellent: 0.8585
- satisfaisant: 0.8402
- excellente: 0.8323
- top: 0.8265
- qualite: 0.8182
```

```
Search in English:

Results for 'customer service':
- deplorable: 0.7984
- sales: 0.7579
- relations: 0.7271
- reachable: 0.7263
- nonexistent: 0.7229

Results for 'high price':
- compared: 0.9200
- deductibles: 0.9000
- guarantees: 0.8981
- franchises: 0.8914
- competitors: 0.8644

Results for 'good quality':
- excellent: 0.9279
- satisfactory: 0.8808
- impeccable: 0.8722
- top: 0.8712
- perfect: 0.8518
```

Pour la recherche sémantique, on constate des résultats cohérents pour les deux langues :

- Pour le « service client », on retrouve des mots comme déplorable, médiocre ou joignable qui sont en adéquation avec le concept du « service client »
- Même interprétation pour « prix élevé » et « bonne qualité »

2.2 Modèle de prédiction de la note

- Modèle basé sur les avis (français et anglais), les assureurs, les produits et la langue (ex : si l'avis renseigné est en anglais, se baser par rapport aux avis anglais pour la prédiction)

PREPARATION DU MODELE

- Encodage de mes variables catégorielles (assureur et produit) :
- Tokenization de mes deux variables d'avis regroupé dans un seul dictionnaire indexé
- On convertit ensuite les textes en deux séquences numériques (une pour le français, l'autre pour l'anglais) et création d'un index pour la langue (1 français, 0 anglais)
- Enfin, on combine les données françaises et anglaises pour l'entraînement

CREATION DU MODELE :

Quatre branches créées :

- Branche texte : Crée une entrée pour le texte (l'avis) avec une taille maximale défini au préalable (MAX_WORDS)
  Transforme les mots en vecteurs de dimension 32 via une couche **d'embedding**
  Applique deux couches **LSTM** (64 puis 32 unités) avec régularisation L2 (0.03)
  Utilise des couches Dropout (0.5 et 0.4) pour prévenir le surapprentissage

- Branche Assureur et Produit : Crée une entrée pour l'assureur et le produit(sous forme de scalaire)
  Transforme les IDs d'assureurs en vecteurs de dimension 4 (2 pour les produits)
  Aplatit le résultat pour la concaténation

- Branche Langue : Prend en entrée l'indicateur de langue (0 pour anglais ou 1 pour français)
  Applique une transformation dense vers 2 dimensions

Fusion et Couches Denses :

- Concatène toutes les features des différentes branches
- Applique deux couches denses (32 puis 16 neurones)
- Utilisation de la régularisation L2 (0.03) et Dropout (0.4 et 0.3) pour prévenir le surapprentissage
- Termine par une couche de sortie linéaire pour la prédiction

COMPILATION ET ENTRAÎNEMENT DU MODELE :

Compilation :

- Optimiseur Adam avec learning rate faible (0.0003) (régulé pour éviter le surapprentissage)
- Fonction de perte MSE

Entraînement :

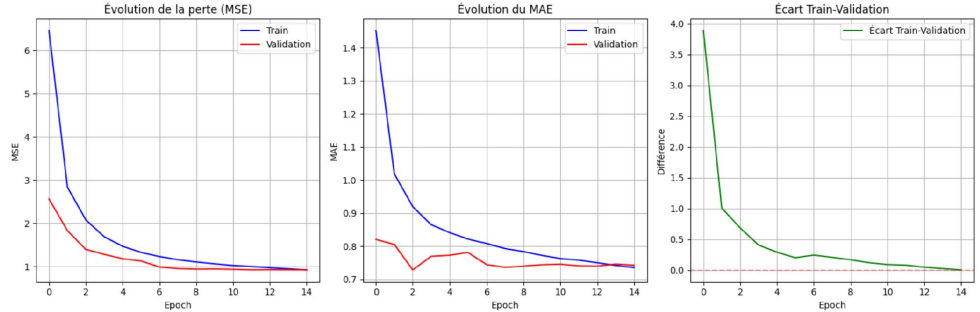
- 20 epochs maximum
- Batch size de 64
- 25% des données pour la validation
- early stopping avec patience de 3 et delta minimum de 0.003 (toujours pour éviter le surapprentissage)

POURQUOI AVOIR CHOISI LSTM :

- Plus performant que RNN (problème de disparition du gradient)
- Comprend le contexte et les relations entre les mots
- LSTM traite les mots dans leur ordre d'apparition
- Bien adapté pour la prédiction de notes (tâche de régression)

PERFORMANCE DU MODELE :

```
Epoch 15/20
565/565 ━━━━━━━━━━━━━━━━━━━━ 7s 13ms/step - loss: 0.9200 - mae: 0.7317 - mse: 0.8390 - val_loss: 0.9240 - val_mae: 0.7426 - val_mse: 0.846
```




```
Analyse du surapprentissage :
Métriques finales :
Loss (Train) : 0.9251
Loss (Validation) : 0.9240
MAE (Train) : 0.7355
MAE (Validation) : 0.7426

Pourcentage de surapprentissage : 0.12%
Meilleure epoch : 15
Nombre total d'epochs effectuées : 15
```

- Surapprentissage très bien géré (**0,12%**) avec l'early stopping, les dropout, les régularisations et le learning rate
- Bonnes performances maintenues (seulement **0,92** de perte sur le jeu d'entraînement et de validation // **0,74** pour la MAE ou erreur absolue moyene)
- A appliquer maintenant sur notre jeu de test, **df_test**

```
323/323 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step
323/323 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step
   Note_fr  Note_en                                               avis  \
0        4        4  Satisfait des tarif et rapide j'attends de voi...
1        4        3  Bon service, service réactif. Tout en ligne pr...
2        5        4  L’assurance Olivier est une très bonne assuran...
3        4        3  Je suis satisfait de l'olivier assurances pas ...
4        4        2  le prix est attractif par rapport à ma situati...

                                              avis_en
0  Satisfied with the price and fast I expect to ...
1                                          Loading...
2  Olivier insurance is a very good insurance I a...
3                                          Loading...
4  The price is attractive compared to my situati...


MSE entre les prédictions françaises et anglaises : 0.7539
```

En appliquant le modèle sur notre base **df_test**, on observe un écart dans les notes prédites pour les avis en français et en anglais (**MSE de 0,75**) sur une base de 10 330 avis.

Cet écart s'explique principalement par des erreurs de traduction, comme le montre le deuxième avis de la base de données. En conséquence, le modèle ne parvient pas à prédire la même note pour les deux langues. Je précise "principalement" car, malgré tout, des différences sont observées dans les notes prédites même pour des avis correctement traduits (comme le troisième et le cinquième avis), ce qui indique que le modèle n'est pas parfait et ne prédit pas systématiquement la même note pour les deux langues.

Cependant, de manière générale, le modèle prédit assez bien les notes dans les deux langues sans faire de grandes distinctions entre les deux langues.

3 Streamlit Application

3.1 Mise en place de l’application

L’application contient :

- **Deux sélecteurs : un pour choisir l’assureur, l’autre le produit**
- **Une interface pour écrire un avis (en français ou anglais)**
- **La note prédite**
- **La catégorie de l’avis**
- **L’explication de la note prédite**
- **Les avis similaires à l’avis écrit**
- **Section Question & Réponse**

MODELES CHOISIS :

- **BART-large-CNN** pour les explications car il génère des textes cohérents et à un bon équilibre entre qualité et rapidité.

- **all-MiniLM-L6-v2** pour la recherche d'avis similaire en créant des embeddings. Il est multilingue et très optimisé pour la similarité sémantique.

- **FLAN-T5-large** pour la section Question & Réponse car il est très performant sur les tâches de génération, étant donné que je demande dans le prompt de générer une réponse directe basée sur l'avis, en utilisant naturellement les mots de la question. Il est donc capable de suivre des instructions complexes et possède un bon support multilingue.

- **BART-large-mnli** pour la classification de la catégorie de l'avis car il est performant pour la classification sans exemples, il est excellent pour comprendre les relations sémantiques et est capable de gérer des textes longs comme des avis clients. Il possède également un support multilingue intégré. La classification **zero-shot** est utilisé globalement pour les mêmes raisons, à savoir qu'il n'a pas besoin d'entraînement sur des données spécifiques et fonctionne très bien en français et en anglais.

L'utilisation de **RoBERTa-base-squad2** était au départ plus préconisé pour la section Question & Réponse car c'est une référence dans la matière mais les premiers tests n'étaient pas concluant et ne retournait pas le résultat attendu.

FONCTIONS PRINCIPALES :

- **prepare_text_for_prediction** qui prépare le texte pour la prédiction du modèle
- **get_explanation** qui génère des explications contextualisées basées sur les mots les plus fréquents retrouvés dans les avis de l'assureur et du produit sélectionné, par rapport à la langue détectée et à la note prédite
- **retrieve_similar_reviews** qui trouve les avis similaires grâce au modèle vu précédemment
- **answer_question** pour répondre aux questions sur l'avis renseigné, en cherchant à réutiliser les mots de la question et de l'avis pour formuler sa réponse

3.2 Fonctionnement de l’application

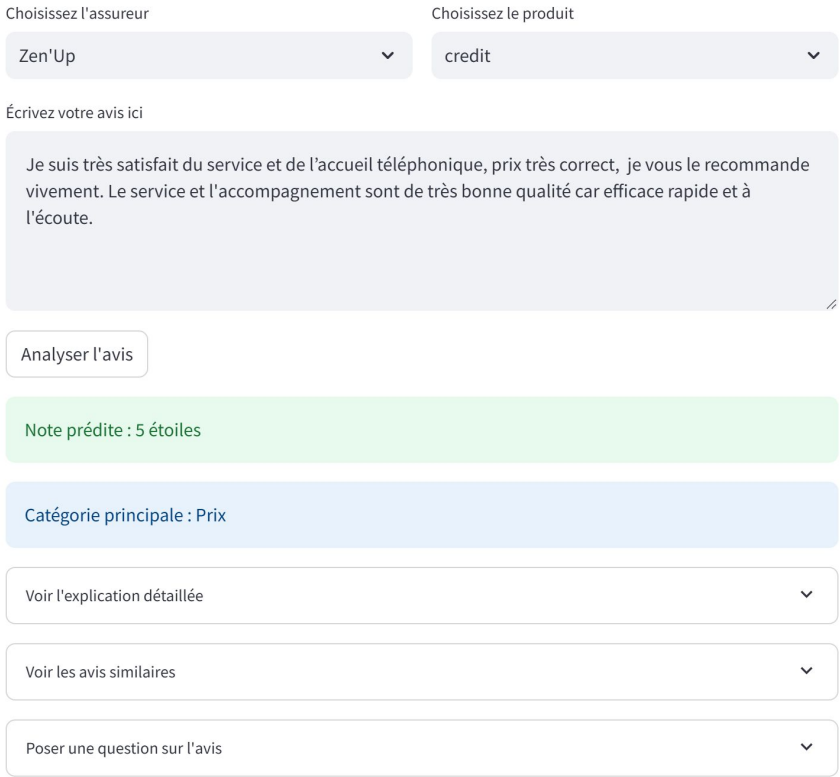
L’utilisateur arrive sur l’interface de l’application :

- **Il sélectionne l’assureur et le produit souhaité**
- **Il renseigne un avis (en français ou en anglais sinon une erreur s’affiche)**

Le modèle se charge alors de prédire la note de l’avis, de le catégoriser, de fournir des explications sur la prédiction et de retrouver des avis similaires

Pour finir, une section Question&Réponse est disponible pour poser une question sur l’avis renseigné et obtenir une réponse claire et concise.

Prédiction et Analyse d'Avis d'Assurance

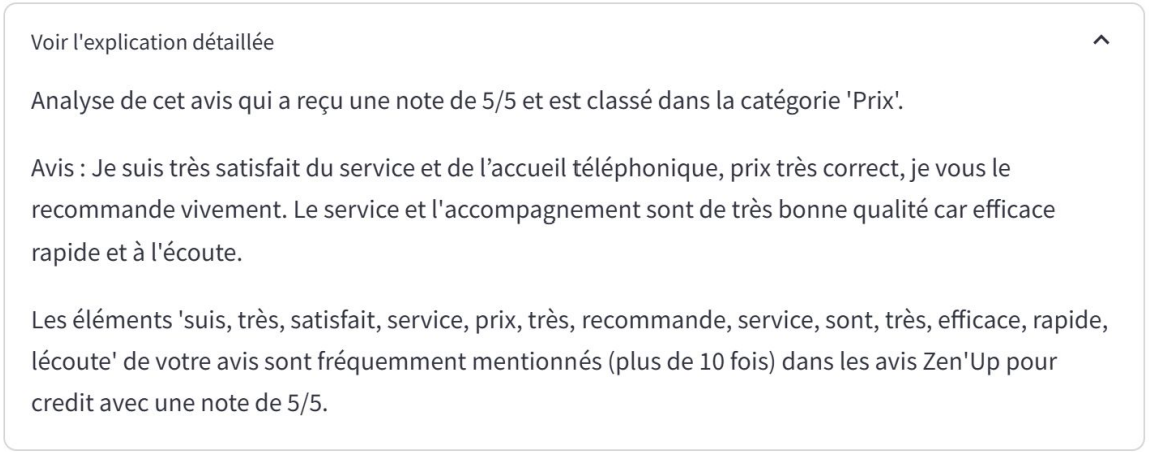


On prend l'exemple de Zen'Up dans les assurances crédits qui contient beaucoup d'avis notés 5/5 (moyenne à 4,44 comme vu précédemment) :

On renseigne un avis avec beaucoup de mots qui se retrouvent souvent dans les avis notés 5/5 comme « très », « satisfait », « prix », « rapide » ou bien « qualité ».

Le modèle prédit bien une note de 5 et catégorise l'avis dans la catégorie « Prix », en retrouvant le terme « prix » dans l'avis.

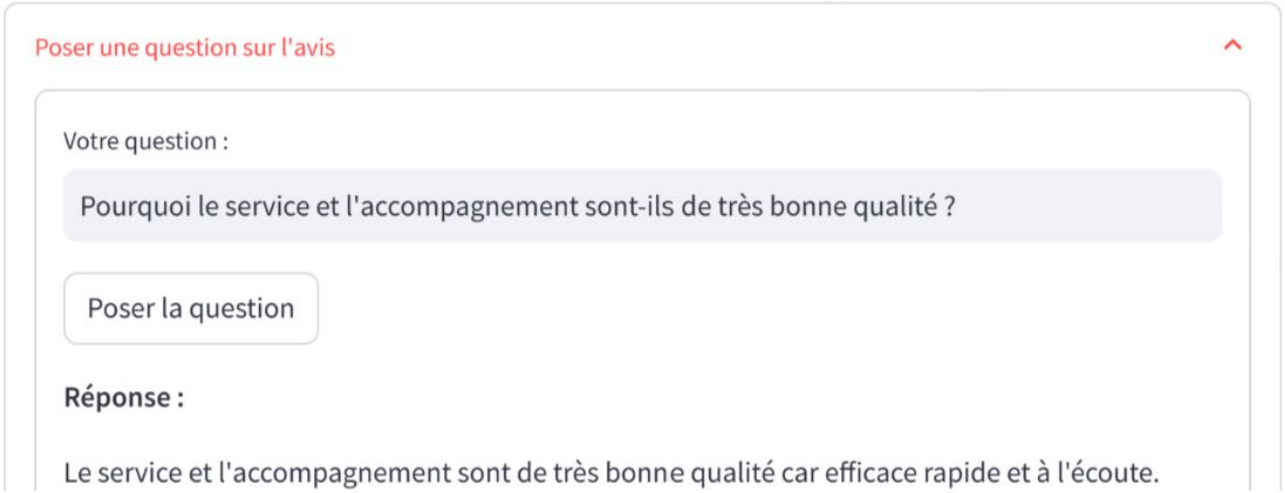
Parmi les catégories suivantes : "Prix", "Couverture d'assurance", "Inscription", "Service client", "Traitement des réclamations", "Annulation" et "Autre" il s'agit de la catégorie la plus logique avec celle du service client potentiellement.



On retrouve ici les explications associées à l'avis, en parcourant bien les avis de l'assureur et du produit choisit, par rapport à la note prédite et à la langue détectée.



On retrouve ici les avis similaires de l'avis renseigné grâce à la similarité cosine, qui calcule les vecteurs des avis les plus proches de l'avis renseigné (les avis sont transformés au préalable en vecteurs avec la fonction vu précédemment)



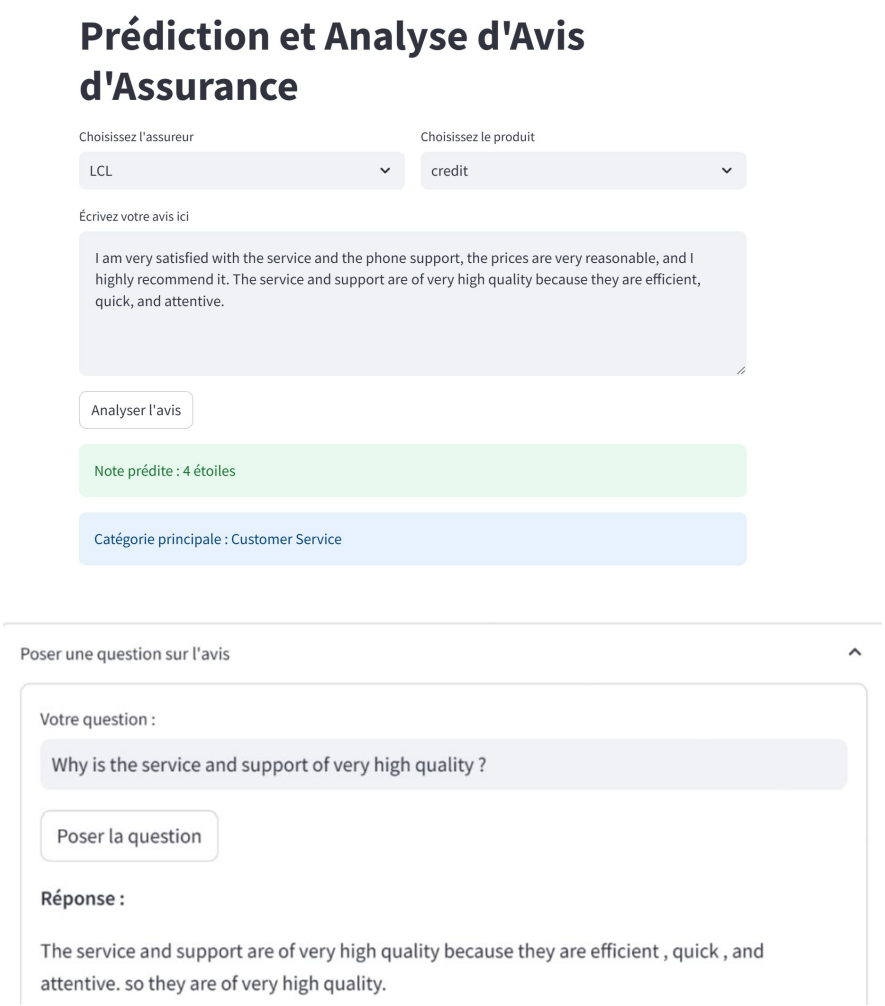

```
Génère une réponse directe et factuelle basée uniquement sur l'avis, en utilisant naturellement les mots de la question.
"""
```

Enfin, ici l'avis répond parfaitement comme l'instruction du prompt l'exige, à savoir une réponse directe basé sur l'avis en reprenant naturellement les mots de la question.



Pour montrer la dépendance du choix de l'assureur et du produit, j'ai décidé de faire un test en gardant le même avis ainsi que le même produit mais avec un assureur différent.

Comme nous pouvons le constater, la note prédite passe à 4 étoiles et cela car LCL ne contient pas d'avis 5 étoiles sur son assurance crédit (alors que Zen'Up en a 149). De ce fait, même si l'avis est très mélioratif, il ne pourra pas atteindre la note de 5 étoiles dans ce cas de figure.



Pour finir, une petite démonstration avec la langue anglaise pour montrer que les modèles s’adaptent bien à la langue

RESSENTI DU PROJET :

J’ai trouvé ce projet très enrichissant, avec la possibilité de découvrir pleins de nouveaux modèles ainsi que la création d’une application Streamlit, une première pour moi.

Le but était de fournir un modèle de prédiction complet et fiable, répondant aux exigences demandées et une application fonctionnelle et je pense que c’est une mission réussie.

La rigueur est très importante, notamment sur le nettoyage des données pour fournir un modèle fiable sur lequel faut éviter le surapprentissage. Une analyse approfondie et plusieurs tests sont nécessaires afin d’optimiser au mieux ses résultats.

LIEN COLAB :

https://colab.research.google.com/drive/1j8de1JPGZf5VLIbSlnQoSzxaHq6HyH7H?usp=sharing